
REQUIREMENTS NOT MET

None

PROBLEMS ENCOUNTERED

None Encountered

FUTURE WORK/APPLICATIONS

External Bus Interfaces are used to use memory devices and expand our memory capacity for the microprocessor we are utilizing. This is paramount for modular storage devices like an SSD or hard drive, to further the storage capacity of our devices. This also lets us utilize a larger range for I/O rather than restricting us to the pre-routed ones available. Pretty cool! 😊

PRE-LAB EXERCISES

- i. For each SRAM configuration within the EBI system of the ATxmega128A1U microcontroller, to which address lines do you have external, physical access?

Without ALE1/2: Utilizes A[21:0] with constant direct external access to all pins.

ALE1: Utilizes A[19:0] with constant direct external access to A[19:16], A[15:8]/A[7:0] are multiplexed.

ALE2: Utilizes A[23:0] with constant direct external access to A[15:8], A[23:16]/A[7:0] are multiplexed.

ALE12: Utilizes A[23:0], where all inputs are multiplexed, no external access.

LPC ALE1: Utilizes A[19:0], where A[7:0] shares a bus with D[7:0], so we have constant direct external access to A[19:8].

LPC ALE12: Utilizes A[19:0], where A[7:0], A[15:8], D[7:0] share a bus, so we have constant direct external access to A[19:16].

NOALE 4PORT: Access to all bits A[23:0].

- ii. For the SRAM LPC 3-PORT ALE1 configuration, is it possible to have external, physical access to any address lines above A15? Why or why not?

Yes, because they are not multiplexed, we can reliably read them, as lines A[19:16] are dedicated.

- iii. Describe what performing full address decoding and partial address decoding signifies. Provide examples of both types for [1] an SRAM chip and [2] either an input port or output port.

We know that our modules, whether they be some sort of IO or storage device will occupy set ranges. We know that our CS0-3 are **programmable** to denote a subset of ranges within our overall memory of 0x3000 – 0xFFFFFFF. These can be between 256B and 16MB, where the base address of our chip-selects must be on the edge of a 4KB increment if it is less than or equal to a 4kB size, OR on the edge of an increment equal to the size of the chip select if greater than 4KB.

Say we want our SRAM to start at the beginning of our external memory (0x3000), with a size of 256B, this would mean that we are occupying the range of 0x3000 – 0x30FF, OR:
0011 0000 0000 0000 – 0011 0000 1111 1111.

Since we are occupying the FULL range, we just have to say that our SRAM's CS connects to CS0, and have the write enable go to PH0, and output enable go to PH1.

However, if the module occupies a subset of a range, say 47 addresses within a 256B module, we have to ensure that this is known by the chip select of our IO's tri-state buffer/d-flipflops!

Say we denote CS1 = 0x3100 – 0x31FF = 0011 0001 0000 0000 – 0011 0001 1111 1111.
We only want to use 0x3100-0x312E = 0011 0001 0000 0000 – 0011 0001 0010 1110

Only acknowledging the parts separate from our chip-select: (A7:A0)
0000 0000 – 0010 1110, we can derive a logic equation of:
$$/A7*/A6*/(A5*A4)*/(A3*A2*A1*A0) = E1 \text{ (for simplicity below)}$$

So our chip selects would be /PH6 * E1 * /PH0 (for write) or /PH6 * E1 * /PH1 (for read)

- iv. In theory, how many 8-bit I/O ports could be mapped to the external data memory space of an ATxmega128A1U microcontroller, assuming that at most one port utilizes any particular address range? Explain your answer, utilizing your response from Exercise iii.

Assuming one address range means one chip-select, we are able to natively support four I/O ports, as we have CS0 – CS3, IF we did address decoding we could technically find the amount of bytes from \$3000 - \$FFFFFF, which is effectively a range of \$FFD000, or 16,764,928 bytes.

- v. Assume that an SRAM component with the same size as the one on the OOTB Memory Base has to be added to a different computing system containing an ATxmega128A1U but no OOTB Memory Base (i.e., this is not the OOTB μ PAD computing system), with the first address of the SRAM starting at address **0x1C8000**, instead of the address specified in the “Lab 4 – EBI Pre-Lab Quiz.” Design, on paper, a hardware expansion for this new system, utilizing the same structure of steps laid out in this quiz. Use the same “overall” memory-mapping constraints presented within the quiz.

Appendix Figure 3, We find that 0x1C8000 rests on a 32k edge, so we do not need to do decoding. Simply, we use CS0 for our 32k ram, and provide A[15:0] through a latch and the data bus.

- vi. Assume that some external 256 KB SRAM must be fully mapped, i.e., fully address decoded to the data memory space of the ATxmega128A1U, with the first memory location of the SRAM corresponding to data memory address **0x6B0000**. Design, on paper, a hardware expansion for this new system, utilizing the same structure of steps laid out in the relevant pre-lab quiz. Use the same “overall” memory-mapping constraints presented within this quiz. Hint: Consider how many address signals are needed to make the SRAM fully addressable and then consider how you will gain access to all these signals.

Appendix Figure 4, We needed to make sure this was on a chip select edge, which it is not (256k), so we must use a smaller size; after analysis we find it is on a 64kb edge, in retrospect we find CS0 will be used for A16; so we use CS1 = 64k, CS2 = 128k (as we are on the available edge), and CS3 = 64k. We use a latch to access the bits A[15:8].

- vii. Assume that some 8-bit input port should be accessible via the 128 consecutive addresses starting at **0x93C200** within the ATxmega128A1U data memory space. Design, on paper, a hardware expansion for this port, utilizing the same structure of steps laid out in the relevant pre-lab quiz. Use the same “overall” memory-mapping constraints presented within this quiz.

Appendix Figure 5, Not on a 4kb edge ($128 \leq 4k$), so we must start at 0x93C000, to reach the end of our desired range (0x93C2FF) we must use a 512 CS size, which means we are covering the range of 0x93C000 to 0x93C3FF. Decoding derives the decreased range we desire.

- viii. Assume that some external 1 MB SRAM must be fully mapped to the data memory space of the ATxmega128A1U, with the first memory location of the SRAM corresponding to data memory address **0x600000**. Design, on paper, a hardware expansion for this new system, utilizing the same structure of steps laid out in the relevant pre-lab quiz. Use the same “overall” memory-mapping constraints presented within this quiz, except that which is concerning the usage of the SRAM 3-PORT ALE1 EBI configuration. Hint: you might consider the use of some other EBI configuration.

Appendix Figure 6, Utilized ALE12, 0x600000 leaves us on a 1MB edge so we do not have to do any decoding. Use two latches for A[15:8], A[23:16] per Figure 27-6 in AU manual.

- ix. How would one go about utilizing the SRAM 4PORT NOALE mode? (Note: Looking at the Alternate Pin Functions only ports H, J, and K by default can be used with the EBI system. How can the other port be activated)?

You first would Enable 4-PORT mode in the EBI_CTRL register, by setting IFMODE[1:0] to 10 for four-port mode, then change SRMODE[1:0] to 11 for NOALE. We then have to activate the fourth port, being either E or F using EBIADROUT[1:0], we can either choose Port F using 00 or Port E using 01 for a full 8-bit usage. We could also move the CS controls to Port PH/PL/PF/PE if desired using EBICSOUT[1:0] (pins 4-7).

PORTH, PORTJ, and PORTK are used for control signals/data bus/address bus respectively, and we add PORTE or PORTF for our upper bits based on our desired configuration described above.

PSEUDOCODE/FLOWCHARTS

Lab4_2:

.equ SRAM_START_ADDR = 0xC58000

.equ IO_START_ADDR = 0xCE72C0

.include "ATxmega128A1Udef.inc"

Create stack pointer

Org & Jump to main

Call EBI_IO_INIT

Call EBI_INIT

Make a pointer to IO_START_ADDR (three bytes)

LOOP:

Load r16 with X

Store X to r16

Jump to LOOP

DONE:

Jump to DONE

;SUBROUTINES

EBI_IO_INIT:

Push register

Set RE, WE, CS0, CS2 as false (active low)

Set ALE1 as false (active high)

Set relevant PORTH controls as outputs

Set PORTK to outputs

Pop register

return

EBI_INIT:

Push register

Set to 8-bit data bus

Set mode and CS size for CS0

Set base address for CS0

Set mode and CS size for CS2

Set base address for CS2

Pop register

return

Lab4_3a:

```
.equ SRAM_START_ADDR = 0xC58000
```

```
.equ IO_START_ADDR = 0xCE72C0
```

```
.include "ATxmega128A1Udef.inc"
```

```
.include "sram_data.asm"
```

In sram_data.asm create directives for the start and end address, create a data_size to reference

In data memory create a .byte allocation with data_size

In program memory create stack pointer

Org and jump to main

Create a program pointer and a data pointer (Z & Y for increment/decrement)

Use a pointer to keep track of data size (X)

WRITE:

Load the next byte of Z and post-increment

Store to Y and post increment

Subtract one from X

brne WRITE

READ:

Reset pointers to initial values for read loop

Load the next byte of Y and post-increment

Store to PORTK output

Delay_300MS

Subtract 1 from X

brne READ

DONE:

Rjmp DONE

; SUBROUTINES:

EBI_IO_INIT:

Push register

Set outputs as false

Enable outputs & controls

Pop register

Return

EBI_INIT:

Push register

Set 8-bit data bus

Set CS size and mode for CS0

Set CS size and mode for CS2

Pop register

Return

TC_INIT:

Push register

Store period to TCC0_PER(+1)

Store clock prescaler

Pop register

Return

DELAY_300MS:

Push register

Clear OVFIF flag

OVF_CHECK:

Check flag

Skip line if OVFIF is set:

Rjmp OVFIF_CHECK

Pop Register

Return

Lab4_3b:

.equ SRAM_START_ADDR = 0xC58000

.equ IO_START_ADDR = 0xCE72C0

.equ DATA_VAL = 0x04

.equ DATA_VAL = 0x07

.include "ATxmega128A1Udef.inc"

Create Stack Pointer

Org and jump to main

MAIN:

Rcall EBI_IO_INIT

Rcall EBI_INIT

Load Y with SRAM_START_ADDR

LOOP:

Load Y with DATA_VAL

Load Y+1 with DATA_VAL2 (use displacement)

Load a register with the data value

Load another register with the second data value

Rjmp LOOP

DONE:

Rjmp DONE

; SUBROUTINES:

EBI_IO_INIT:

Push register

Set outputs as false

Enable outputs & controls

Pop register

Return

EBI_INIT:

Push register

Set 8-bit data bus

Set CS size and mode for CS0

Set CS size and mode for CS2

Pop register

Return

PROGRAM CODE

Lab4_2:

```
;
; Lab4.asm
;
; Created: 6/16/2026 4:39:45 PM
; Author : Evan Baesler
;

.equ SRAM_START_ADDR = 0xC58000
.equ IO_START_ADDR = 0xCE72C0

.include "ATxmega128A1Udef.inc"

ldi r16, low(RAMEND) ; Stack Pointer initialization
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

.cseg
.org 0x0100
rjmp MAIN

MAIN:
rcall EBI_IO_INIT
rcall EBI_INIT

ldi XL, low(IO_START_ADDR) ; Point to IO Start
ldi XH, high(IO_START_ADDR)
ldi r16, byte3(IO_START_ADDR)
out CPU_RAMPX, r16

LOOP:
ld r16, X ; Load X (/RE enabled)
st X, r16 ; Store X (/WE enabled)

rjmp LOOP

DONE:
rjmp DONE

; SUBROUTINES:
EBI_IO_INIT:
push r16 ; Preserve Register

ldi r16, 0b01010011
sts PORTH_OUTSET, r16 ; Sets RE, WE, CS0, CS2 as false.

ldi r16, (1<<2)
sts PORTH_OUTCLR, r16 ; Sets ALE1 as an false to be output from the XMega.

ldi r16, 0b01010111 ; Initializes outputs for WE,RE,ALE1,CS0,CS2
sts PORTH_DIRSET, r16

ldi r16, 0xFF
sts PORTK_DIRSET, r16 ; Sets addresses to be output from the microcontroller.
```

```
pop r16  
ret
```

```
EBI_INIT:  
push r16
```

```
ldi r16, 1 ; 8-bit data bus  
sts EBI_CTRL, r16 ; EBI CTRL IFMODE[1:0], AU 27.12 (VERY USEFUL)  
  
; CS0 SETUP  
ldi r16, 0b00011101 ; MODE[1:0], ASIZE[4:0], 27.11 AU  
                        ; SRAM, 32Kb  
sts EBI_CS0_CTRLA, r16  
ldi r16, byte2(SRAM_START_ADDR) ; Load the upper bytes base address.  
sts EBI_CS0_BASEADDR, r16  
ldi r16, byte3(SRAM_START_ADDR)  
sts EBI_CS0_BASEADDR+1, r16  
  
; CS2 SETUP  
ldi r16, 0b00001001 ; MODE[1:0], ASIZE[4:0], 27.11 AU  
                        ; SRAM, 1Kb  
sts EBI_CS2_CTRLA, r16  
ldi r16, byte2(IO_START_ADDR) ; Load the upper bytes base address.  
sts EBI_CS2_BASEADDR, r16  
ldi r16, byte3(IO_START_ADDR)  
sts EBI_CS2_BASEADDR+1, r16  
  
pop r16  
ret
```

Lab4_3a:

```
/*
 * lab4_3a.asm
 *
 * Created: 6/16/2026 6:26:23 PM
 * Author: Evan Baesler
 */

/*
 * lab4_3.asm
 *
 * Created: 6/16/2026 7:28:55 PM
 * Author: Evan Baesler
 */
;

.equ SRAM_START_ADDR = 0xC58000
.equ IO_START_ADDR = 0xCE72C0

.include "ATxmega128A1Udef.inc"
.include "sram_data.asm"

.dseg
.byte DATA_SIZE

.cseg
ldi r16, low(RAMEND) ; Set up Stack Pointer (SP)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

.cseg
.org 0x01000
rjmp MAIN

MAIN:
rcall EBI_IO_INIT
rcall EBI_INIT
rcall TC_INIT

; Setup Flash Pointer (Z) and SRAM Pointer (Y)
ldi ZL, low(DATA << 1) ;
ldi ZH, high(DATA << 1)

ldi YL, low(SRAM_START_ADDR)
ldi YH, high(SRAM_START_ADDR)
ldi r16, byte3(SRAM_START_ADDR)
out CPU_RAMPY, r16 ; Needed for > 64KB addressing, 2^16

ldi XL, low(DATA_SIZE)
ldi XH, high(DATA_SIZE) ; Count size of data

WRITE_LOOP:

lpm r16, Z+ ; Load byte from Program Memory
st Y+, r16 ; Store byte to External SRAM
sbiw X, 1 ; Decrement Counter
brne WRITE_LOOP
```

```
; Reset pointers for Read Loop
ldi YL, low(SRAM_START_ADDR)
ldi YH, high(SRAM_START_ADDR)
ldi r16, byte3(SRAM_START_ADDR)
out CPU_RAMPY, r16

ldi XL, low(DATA_SIZE)
ldi XH, high(DATA_SIZE)

; Read SRAM data and output to Port K (LEDs)
READ_LOOP:
ld r17, Y+ ; Read from External SRAM
sts PORTK_OUT, r17 ; Send to IO port
rcall DELAY_300MS ; Wait 300ms
sbiw X, 1 ; Subtract 1 from Counter
brne READ_LOOP

DONE:
rjmp DONE

; --- SUBROUTINES ---

EBI_IO_INIT:
push r16

ldi r16, 0b01010011 ; Sets active-low outputs as false (RE, WE, CS0/2)
sts PORTH_OUTSET, r16
ldi r16, (1<<2)
sts PORTH_OUTCLR, r16 ; Sets active-high outputs as false (ALE1)
ldi r16, 0b01010111
sts PORTH_DIRSET, r16 ; Enables control outputs
ldi r16, 0xFF
sts PORTK_DIRSET, r16 ; Enables outputs

pop r16
ret

EBI_INIT:
push r16

ldi r16, 1 ; 8-bit data bus
sts EBI_CTRL, r16 ; AU 27.12, IFMODE[1:0]

; CS0 SETUP
ldi r16, 0b00011101 ; MODE[1:0], ASIZE[4:0], 27.11 AU
; SRAM, 32Kb

sts EBI_CS0_CTRLA, r16
ldi r16, byte2(SRAM_START_ADDR) ; Load base address to upper bytes.
sts EBI_CS0_BASEADDR, r16
ldi r16, byte3(SRAM_START_ADDR)
sts EBI_CS0_BASEADDR+1, r16

; CS2 SETUP
ldi r16, 0b0001001 ; MODE[1:0], ASIZE[4:0], 27.11 AU
; SRAM, 32Kb

sts EBI_CS2_CTRLA, r16
ldi r16, byte2(IO_START_ADDR) ; Load base address to upper bytes.
sts EBI_CS2_BASEADDR, r16
ldi r16, byte3(IO_START_ADDR)
```

```
sts EBI_CS2_BASEADDR+1, r16
```

```
pop r16  
ret
```

```
TC_INIT:  
push r16
```

```
; SCF = 2MHz, PRE = 64, D = 300*10^-3, PER = 9375 Ticks  
ldi r16, low(9375)  
sts TCC0_PER, r16  
ldi r16, high(9375)  
sts TCC0_PER+1, r16 ; Set period of 47ms
```

```
ldi r16, TC_CLKSEL_DIV64_gc  
sts TCC0_CTRLA, r16 ; Start counting with DIV64 prescaler, AU 14.12
```

```
pop r16  
ret
```

```
DELAY_300MS:  
push r16
```

```
ldi r16, TC0_OVFIF_bm  
sts TCC0_INTFLAGS, r16 ; Resets flag (stores 1 to flag)
```

```
OVF_CHECK:  
lds r16, TCC0_INTFLAGS ; Loads flags  
sbrs r16, TC0_OVFIF_bp ; Checks bit designated for OVFIF flag  
rjmp OVF_CHECK
```

```
pop r16  
ret
```


Lab4_3b:

```
/*
 * lab4_3.asm
 *
 * Created: 6/16/2026 7:28:55 PM
 * Author: Evan Baesler
 */
;

.equ SRAM_START_ADDR = 0xC58000
.equ IO_START_ADDR = 0xCE72C0
.equ DATA_VAL = 0x04
.equ DATA_VAL2 = 0x07 ; :)

.include "ATxmega128A1Udef.inc"

ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

.cseg
.org 0x0100
rjmp MAIN

MAIN:
rcall EBI_IO_INIT
rcall EBI_INIT
rcall TC_INIT

ldi YL, low(SRAM_START_ADDR)
ldi YH, high(SRAM_START_ADDR)
ldi r16, byte3(SRAM_START_ADDR)
out CPU_RAMPY, r16

LOOP:
ldi r16, DATA_VAL
st Y, r16 ; Stores base address

ldi r16, DATA_VAL2
std Y+1, r16 ; Stores by displacement, looping thru same addresses

ld r17, Y ; Loads base address
ldd r18, Y+1 ; Loads by displacement, looping thru same addresses

rjmp LOOP

DONE:
rjmp DONE

; SUBROUTINES:
EBI_IO_INIT:
push r16

ldi r16, 0b01010011 ; Sets active-low outputs as false (RE, WE, CS0/2)
sts PORTH_OUTSET, r16
ldi r16, (1<<2)
sts PORTH_OUTCLR, r16 ; Sets active-high outputs as false (ALE1)
ldi r16, 0b01010111
```

```
sts PORTH_DIRSET, r16 ; Enables control outputs
ldi r16, 0xFF
sts PORTK_DIRSET, r16 ; Enables outputs

pop r16
ret

EBI_INIT:
push r16

ldi r16, 1 ; 8-bit data bus
sts EBI_CTRL, r16 ; IFMODE[1:0], AU 27.12

; CS0 SETUP
ldi r16, 0b00011101 ; MODE[1:0], ASIZE[4:0], 27.11 AU
                        ; SRAM, 32Kb
sts EBI_CS0_CTRLA, r16
ldi r16, byte2(SRAM_START_ADDR)
sts EBI_CS0_BASEADDR, r16
ldi r16, byte3(SRAM_START_ADDR)
sts EBI_CS0_BASEADDR+1, r16

; CS2 SETUP
ldi r16, 0b00001001 ; MODE[1:0], ASIZE[4:0], 27.11 AU
                        ; SRAM, 32Kb
sts EBI_CS2_CTRLA, r16
ldi r16, byte2(IO_START_ADDR)
sts EBI_CS2_BASEADDR, r16
ldi r16, byte3(IO_START_ADDR)
sts EBI_CS2_BASEADDR+1, r16

pop r16
ret

TC_INIT:
push r16

; SCF = 2MHz, PRE = 64, D = 47*10^-3, PER = 11750 Ticks
ldi r16, low(9375)
sts TCC0_PER, r16
ldi r16, high(9375)
sts TCC0_PER+1, r16 ; Set period of 47ms

ldi r16, TC_CLKSEL_DIV64_gc ; AU 14.12
sts TCC0_CTRLA, r16 ; Start counting with DIV64 prescaler

pop r16
ret

DELAY_300MS:
push r16

ldi r16, TC0_OVFIF_bm
sts TCC0_INTFLAGS, r16 ; Resets flag (stores 1 to flag)

OVF_CHECK:
lds r16, TCC0_INTFLAGS ; Loads flag
sbrs r16, TC0_OVFIF_bp ; Checks bit designated for OVFIF flag
rjmp OVF_CHECK
```

```
pop r16  
ret
```

APPENDIX

Word Address			
ATxmega128A1U		ATxmega64A1U	
0		0	Application Section (bytes) (128K/64K)
			...
FFFF	/	77FF	Application Table Section (bytes) (8K/ 4K)
F000	/	7800	
FFFF	/	7FFF	
10000	/	8000	Boot Section (bytes) (8K/4K)
10FFF	/	87FF	

Figure 1: Program Memory Addressing

Byte Address	ATxmega64A1U	Byte Address	ATxmega128A1U
0	I/O Registers (4K)	0	I/O Registers (4KB)
FFF		FFF	
1000	EEPROM (2K)	1000	EEPROM (2K)
17FF		17FF	
	RESERVED		RESERVED
2000	Internal SRAM (4K)	2000	Internal SRAM (8K)
2FFF		3FFF	
4000	External Memory (0 to 16MB)	3000	External Memory (0 to 16MB)
FFFFFF		FFFFFF	

Figure 2: Data Memory Addressing

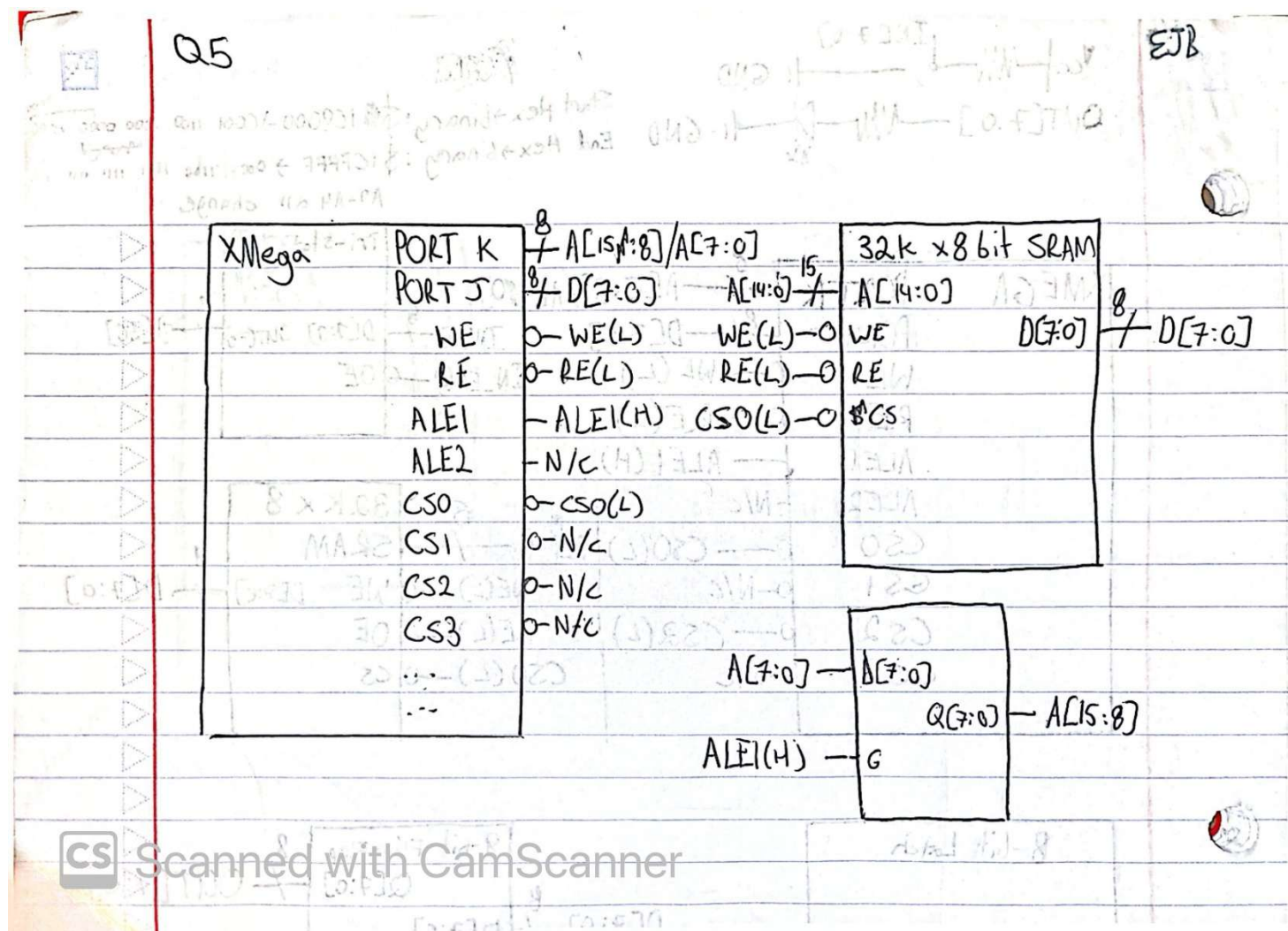


Figure 3: Work for Lab Question 5 (32kb RAM at starting at 0x1C8000)

1011
0110 0000 0000 0000 0000
0110 1111 0000 0000 0000

CS1 = 64K (680000 edge)
CS2 = 128K (6C0000 edge)
CS3 = 64K (6E0000 edge)

Q16

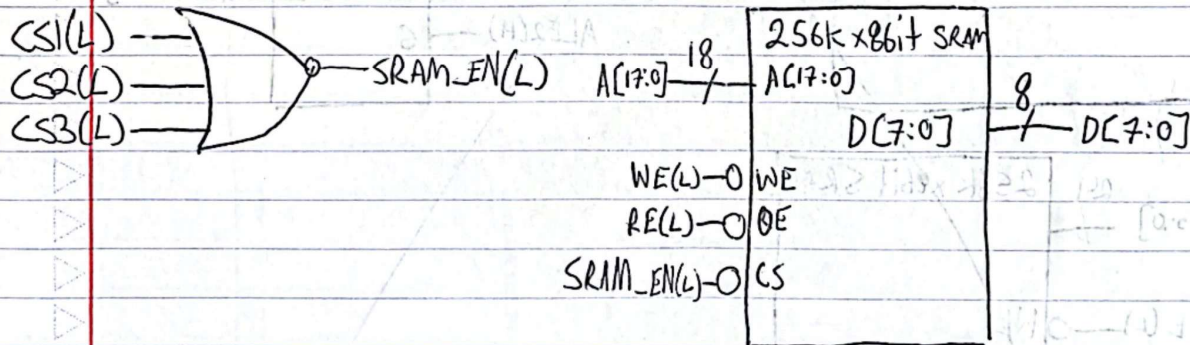
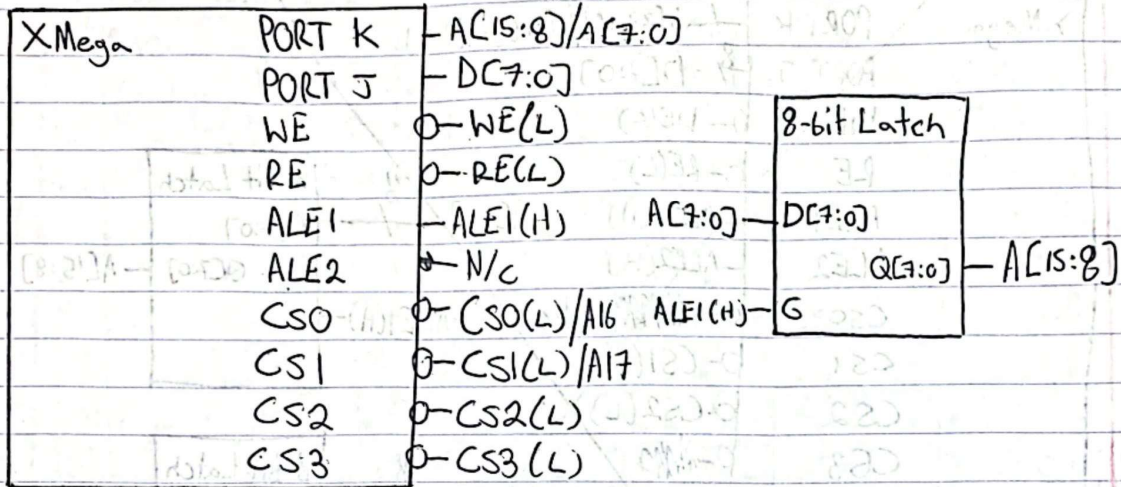
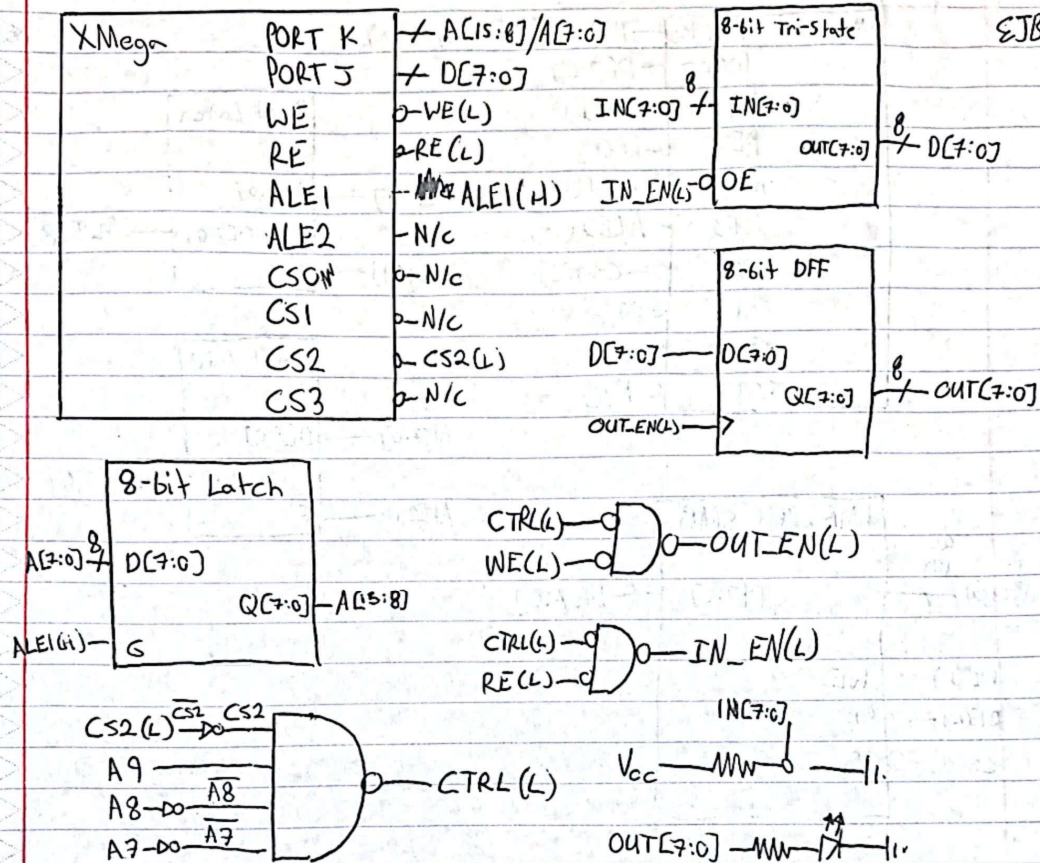


Figure 4: Work for Lab Question 6.

Start: 0x93C200 - 0x93C2FF

$128 \leq 4k$, must start at 4k boundary, nearest is 0x93C000
must use CS0 = 512k to reach.

Q7



Decode

1001 0011 1100 0010 0000 0000 - 1001 0011 1100 0010 0111 1111

98 7654 98 7654

$\overline{IN_EN} = \overline{WE} * \overline{A9} * \overline{A8} * \overline{A7} * \overline{CS2}$

$\overline{OUT_EN} = \overline{RE} * \overline{A9} * \overline{A8} * \overline{A7} * \overline{CS2}$

Figure 5: Work for Lab Question 7.

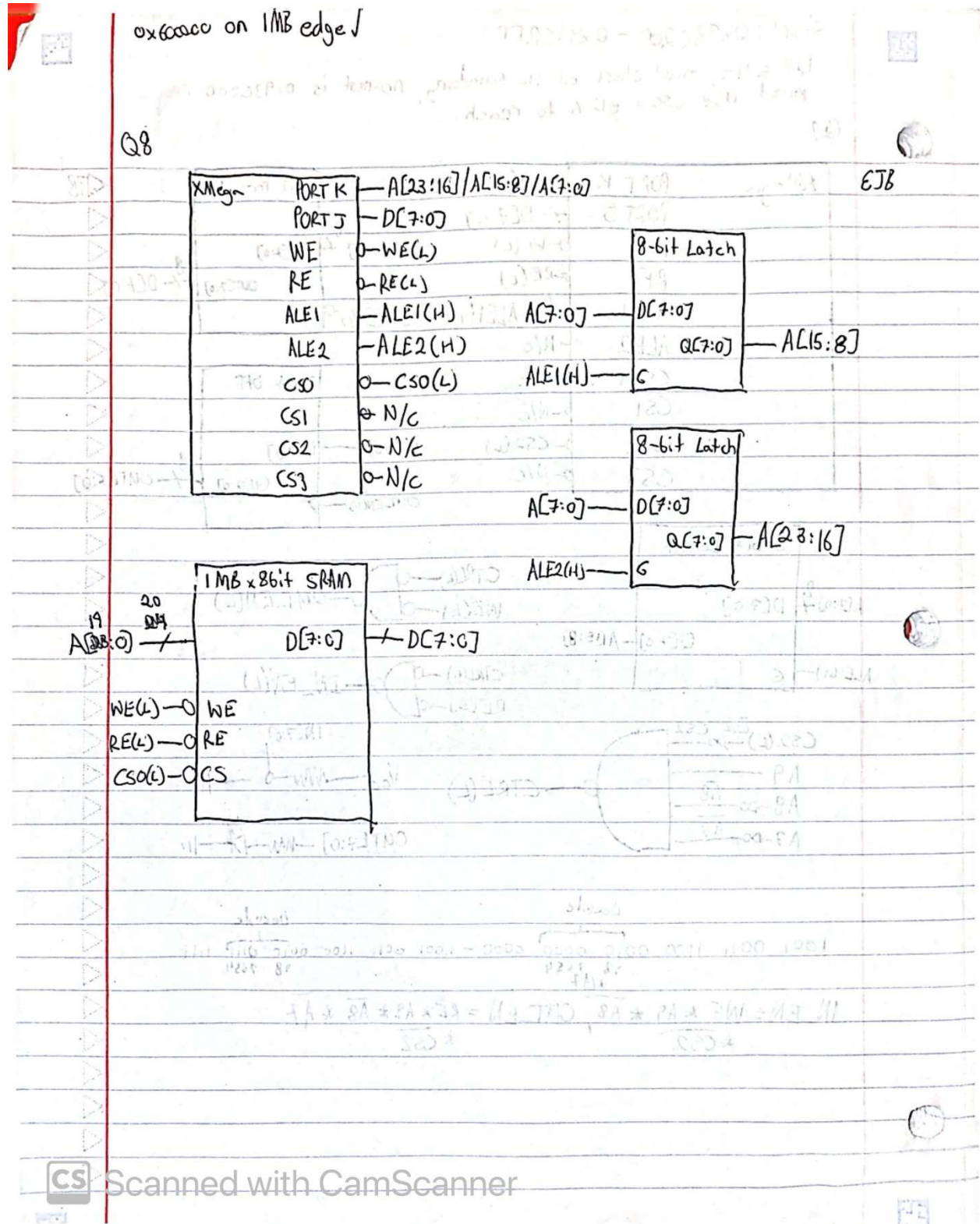


Figure 6: Work for Lab Question 8.

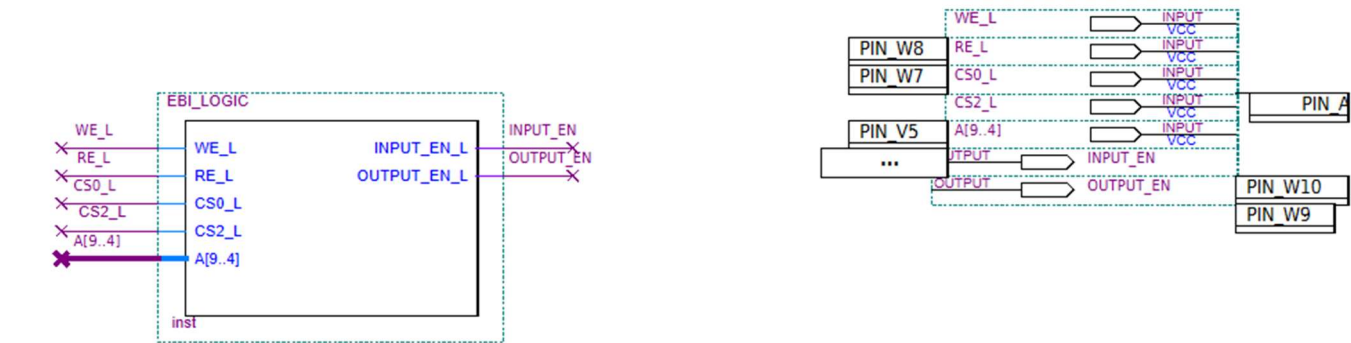
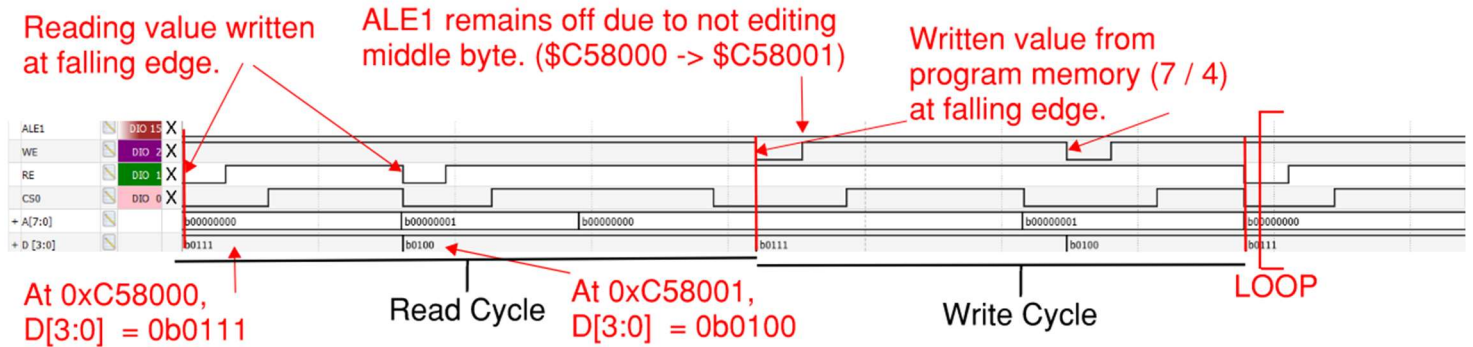


Figure 7: Quartus BDF

```
1  library ieee; use ieee.std_logic_1164.all;
2
3  entity EBI_LOGIC is port(
4
5      -- We have our CS enables and RE/WE
6      WE_L: in std_logic;
7      RE_L: in std_logic;
8      CS0_L: in std_logic;
9      CS2_L: in std_logic;
10     A: in std_logic_vector(9 downto 4);
11
12     INPUT_EN_L: out std_logic;
13     OUTPUT_EN_L: out std_logic);
14
15  end EBI_LOGIC;
16
17  architecture behavior of EBI_LOGIC is
18
19     signal WE: std_logic;
20     signal RE: std_logic;
21     signal CS0: std_logic;
22     signal CS2: std_logic;
23     signal ADDRESSING: std_logic;
24     signal INPUT_EN: std_logic;
25     signal OUTPUT_EN: std_logic;
26
27  begin
28
29     WE <= not(WE_L);
30     RE <= not(RE_L);
31     CS0 <= not(CS0_L);
32     CS2 <= not(CS2_L);
33     ADDRESSING <= (A(9) and not A(8) and A(7) and A(6) and (not A(5) or not A(4)));
34     INPUT_EN_L <= not(INPUT_EN);
35     OUTPUT_EN_L <= not(OUTPUT_EN);
36
37     -- Outputs
38     INPUT_EN <= (CS2 and RE and ADDRESSING);
39     OUTPUT_EN <= ((CS2 and WE and ADDRESSING) or (CS0));
40
41  end behavior;
```

Figure 8: Quartus VHDL



When CS0 goes low, the Xmega enables the relevant control (RE/WE) for loading or storing data, where either goes twice in a row for each pair of addresses, accessing (and reading/storing) the relevant value.

Figure 9: Waveforms DIO Capture of Lab4_3b

13.16 Register summary – Ports

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DIR	DIR[7:0]								148
+0x01	DIRSET	DIRSET[7:0]								148
+0x02	DIRCLR	DIRCLR[7:0]								148
+0x03	DIRTGL	DIRTGL[7:0]								148
+0x04	OUT	OUT[7:0]								149
+0x05	OUTSET	OUTSET[7:0]								149
+0x06	OUTCLR	OUTCLR[7:0]								149
+0x07	OUTTGL	OUTTGL[7:0]								149
+0x08	IN	IN[7:0]								150
+0x09	INTCTRL	-	-	-	-	INT1LVL[1:0]		INT0LVL[1:0]		150
+0x0A	INT0MASK	INT0MSK[7:0]								150
+0x0B	INT1MASK	INT1MSK[7:0]								150
+0x0C	INTFLAGS	-	-	-	-	-	-	INT1IF	INT0IF	151
+0x0D	Reserved	-	-	-	-	-	-	-	-	
+0x0E	REMAP	-	-	SPI	USART0	TC0D	TC0C	TC0B	TC0A	151
+0x0F	Reserved	-	-	-	-	-	-	-	-	
+0x10	PIN0CTRL	SRLN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x11	PIN1CTRL	SRLN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x12	PIN2CTRL	SRLN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x13	PIN3CTRL	SRLN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x14	PIN4CTRL	SRLN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x15	PIN5CTRL	SRLN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x16	PIN6CTRL	SRLN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x17	PIN7CTRL	SRLN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x18	Reserved	-	-	-	-	-	-	-	-	
+0x19	Reserved	-	-	-	-	-	-	-	-	
+0x1A	Reserved	-	-	-	-	-	-	-	-	
+0x1B	Reserved	-	-	-	-	-	-	-	-	
+0x1C	Reserved	-	-	-	-	-	-	-	-	
+0x1D	Reserved	-	-	-	-	-	-	-	-	
+0x1E	Reserved	-	-	-	-	-	-	-	-	
+0x1F	Reserved	-	-	-	-	-	-	-	-	

Figure 10: Ports Register Summary

14.13 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRLA	–	–	–	–	CLKSEL[3:0]				175
+0x01	CTRLB	CCDEN	CCCEN	CCBEN	CCAEN	–	WGMODE[2:0]			175
+0x02	CTRLC	–	–	–	–	CMPD	CMPC	CMPB	CMPA	176
+0x03	CTRLD	EVACT[2:0]			EVDLY	EVSEL[3:0]				176
+0x04	CTRLF	–	–	–	–	–	–	BYTEM		178
+0x05	Reserved	–	–	–	–	–	–	–	–	
+0x06	INTCTRLA	–	–	–	–	ERRINTLVL[1:0]		OVINTLVL[1:0]		178
+0x07	INTCTRLB	CCCINTLVL[1:0]		CCBINTLVL[1:0]		CCAINTLVL[1:0]		CCBINTLVL[1:0]		178
+0x08	CTRLFCLR	–	–	–	QDECINDX	CMD[1:0]		LUPD	DIR	179
+0x09	CTRLFSET	–	–	–	QDECINDX	CMD[1:0]		LUPD	DIR	180
+0x0A	CTRLGCLR	–	–	–	CCDBV	CCCBV	CCBBV	CCABV	PERBV	180
+0x0B	CTRLGSET	–	–	–	CCDBV	CCCBV	CCBBV	CCABV	PERBV	180
+0x0C	INTFLAGS	CCDIF	CCCIF	CCBIF	CCAIF	–	–	ERRIF	OVFIF	180
+0x0D	Reserved	–	–	–	–	–	–	–	–	
+0x0E	Reserved	–	–	–	–	–	–	–	–	
+0x0F	TEMP	TEMP[7:0]								181
+0x10 to +0x1F	Reserved	–	–	–	–	–	–	–	–	
+0x20	CNTL	CNT[7:0]								181
+0x21	CNTH	CNT[15:8]								181
+0x22 to +0x25	Reserved	–	–	–	–	–	–	–	–	
+0x26	PERL	PER[7:0]								182
+0x27	PERH	PER[8:15]								182
+0x28	CCAL	CCA[7:0]								182
+0x29	CAAH	CCA[15:8]								182
+0x2A	CCBL	CCB[7:0]								182
+0x2B	CCBH	CCB[15:8]								182
+0x2C	CCCL	CCC[7:0]								182
+0x2D	CCCH	CCC[15:8]								182
+0x2E	CCDL	CCD[7:0]								182
+0x2F	CCDH	CCD[15:8]								182
+0x30 to +0x35	Reserved	–	–	–	–	–	–	–	–	
+0x36	PERBUFL	PERBUF[7:0]								183

Figure 11: Timer/Counter Register Summary

12.9 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	STATUS	NMIEX	–	–	–	–	HILVLEX	MEDLVLEX	LOLVLEX	136
+0x01	INTPRI	INTPRI[7:0]								136
+0x02	CTRL	RREN	IVSEL	–	–	–	HILVLEN	MEDLVLEN	LOLVLEN	137

Figure 12: Interrupts Register Summary

27.12 Register summary – EBI

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRL	SDDATAW[1:0]		LPCMODE[1:0]		SRMODE[1:0]		IFMODE[1:0]		329
+0x01	SDRAMCTRLA	—	—	—	—	SDCAS	SDROW	SDCOL[1:0]		330
+0x02	Reserved	—	—	—	—	—	—	—	—	
+0x03	Reserved	—	—	—	—	—	—	—	—	
+0x04	REFRESHL	SDRAM Refresh Period Low Byte								331
+0x05	REFRESHH	—	—	—	—	—	—	SDRAM Refresh Period High		331
+0x06	INITDLYL	SDRAM Initialization Time Low Byte								331
+0x07	INITDLYH	—	—	SDRAM Initialization Time High Byte						331
+0x08	SDRAMCTRLB	MRDLY[1:0]		ROWCYCDLY[[2:0]			RPDLY[2:0]			332
+0x09	SDRAMCTRLC	WRDLY[1:0]		ESRDLY[2:0]			ROWCOLDLY[2:0]			333
+0x0A	Reserved	—	—	—	—	—	—	—	—	
+0x0B	Reserved	—	—	—	—	—	—	—	—	
+0x0C	Reserved	—	—	—	—	—	—	—	—	
+0x0D	Reserved	—	—	—	—	—	—	—	—	
+0x0E	Reserved	—	—	—	—	—	—	—	—	
+0x0F	Reserved	—	—	—	—	—	—	—	—	
+0x10	CS0	Chip Select 0 Offset Address								
+0x14	CS1	Chip Select 1 Offset Address								
+0x18	CS2	Chip Select 2 Offset Address								
+0x1C	CS3	Chip Select 3 Offset Address								

Figure 13: EBI Register Summary

27.13 Register summary – EBI chip select

Address	Name		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page	
+0x00	CTRLA		–	ASIZE[4:0]					MODE[1:0]		329	
+0x01	CTRLB	(SRAM)	–	–	–	–	–	SRWS[2:0]			330	
		(SDRAM)	SDINITDONE	–	–	–	–	SDSREN	SDMODE[1:0]			
+0x02	BASEADDRL		Chip Select Base Address Low Byte					–	–	–	–	337
+0x03	BASEADDRH		Chip Select Base Address High Byte									337

Figure 14: EBI Chip Select Register Summary

Table 33-7. Port H - alternate functions.

PORT H	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
GND	53							
VCC	54							
PH0	55	SYNC	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$
PH1	56	SYNC	$\overline{CAS}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$
PH2	57	SYNC/ASYN	$\overline{RAS}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$
PH3	58	SYNC	$\overline{DQM}$		$\overline{ALE2}$			$\overline{ALE2}$
PH4	59	SYNC	BA0	$\overline{CS0}/A16$	$\overline{CS0}$	$\overline{CS0}/A16$	$\overline{CS0}$	$\overline{CS0}/A16$
PH5	60	SYNC	BA1	$\overline{CS1}/A17$	$\overline{CS1}$	$\overline{CS1}/A17$	$\overline{CS1}$	$\overline{CS1}/A17$
PH6	61	SYNC	CKE	$\overline{CS2}/A18$	$\overline{CS2}$	$\overline{CS2}/A18$	$\overline{CS2}$	$\overline{CS2}/A18$
PH7	62	SYNC	CLK	$\overline{CS3}/A19$	$\overline{CS3}$	$\overline{CS3}/A19$	$\overline{CS3}$	$\overline{CS3}/A19$

- Notes:
1. CS0 - CS3 can optionally be moved to Port E or F
 2. A16-A23 can optionally be moved to Port E or F when EBI configured in 4PORT mode. Refer to the EBIOUT register description in the XMEGA AU Manual.

Figure 15: PORTH Alternate Functions

Table 33-8. Port J - alternate functions.

PORT J	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
GND	63							
VCC	64							
PJ0	65	SYNC	D0	D0	D0	D0/A0	D0/A0	D0/A0/A8
PJ1	66	SYNC	D1	D1	D1	D1/A1	D1/A1	D1/A1/A9

ATxmega128A1U/ATxmega64A1U

PORT J	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
PJ2	67	SYNC/ASYN	D2	D2	D2	D2/A2	D2/A2	D2/A2/A10
PJ3	68	SYNC	D3	D3	D3	D3/A3	D3/A3	D3/A3/A11
PJ4	69	SYNC	A8	D4	D4	D4/A4	D4/A4	D4/A4/A12
PJ5	70	SYNC	A9	D5	D5	D5/A5	D5/A5	D5/A5/A13
PJ6	71	SYNC	A10	D6	D6	D6/A6	D6/A6	D6/A6/A14
PJ7	72	SYNC	A11	D7	D7	D7/A7	D7/A7	D7/A7/A15

Figure 16: PORTJ Alternate Functions

Table 33-9. Port K - alternate functions.

PORT K	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE2	LPC3 ALE1
GND	73					
VCC	74					
PK0	75	SYNC	A0	A0/A8	A0/A8/A16	A8
PK1	76	SYNC	A1	A1/A9	A1/A9/A17	A9
PK2	77	SYNC/ASYNC	A2	A2/A10	A2/A10/A18	A10
PK3	78	SYNC	A3	A3/A11	A3/A11/A19	A11
PK4	79	SYNC	A4	A4/A12	A4/A12/A20	A12
PK5	80	SYNC	A5	A5/A13	A5/A13/A21	A13
PK6	81	SYNC	A6	A6/A14	A6/A14/A22	A14
PK7	82	SYNC	A7	A7/A15	A7/A15/A23	A15

Figure 17: PORTK Alternate Functions